

```

CREATE TABLE personne(
  id_personne INTEGER PRIMARY KEY,
  nom VARCHAR,
  prenom VARCHAR
);

CREATE TABLE module(
  id_module INTEGER PRIMARY KEY,
  intitule VARCHAR,
  ue VARCHAR,
  code VARCHAR,
  responsable INTEGER REFERENCES personne
);

CREATE TABLE evaluation(
  id_evaluation INTEGER PRIMARY KEY,
  nom VARCHAR,
  dateEval VARCHAR,
  coefficient REAL,
  echelle REAL,
  id_module INTEGER REFERENCES module
);

CREATE TABLE notation(
  id_etudiant INTEGER REFERENCES personne,
  id_evaluation INTEGER REFERENCES evaluation,
  note FLOAT,
  PRIMARY KEY (id_etudiant, id_evaluation)
);

```

SEANCE 3 :

```

--1)
SELECT id_evaluation , nom, dateeval, coefficient, echelle, id_module,
AVG(note) / echelle * 20 AS moyenne_sur_20
FROM notation JOIN evaluation USING(id_evaluation)
GROUP BY id_evaluation, nom, dateeval, coefficient, echelle, id_module
LIMIT 20;

```

--2) Non, il faut aller chercher dans une autres tables les informatiojns.
Une autre table ne peut aller chercher des informations dans une autre table.

```

--3)
CREATE VIEW vue_evaluation AS Q1

```

```

CREATE VIEW vue_evaluation AS SELECT id_evaluation, nom, dateeval, coefficient, echelle, id_module, AVG(note)/echelle * 20 AS
moyenne_sur_20
FROM notation JOIN evaluation USING(id_evaluation)
GROUP BY id_evaluation, nom, dateeval, coefficient, echelle, id_module;

```

```

--4)
EXPLAIN ANALYSE SELECT id_evaluation , nom, dateeval, coefficient, echelle, id_module,AVG(note) / echelle * 20 AS moyenne_sur_20
FROM notation JOIN evaluation USING(id_evaluation)
GROUP BY id_evaluation, nom, dateeval, coefficient, echelle, id_module
LIMIT 20;

```

EXPLAIN ANALYSE SELECT * FROM vue_evaluation; --Voir le plan d'exécution de la requête

--Même chose pour les deux rezquête donc la vue n' pas permis d'optimiser l'opération. Car la vue n'est pas stocké donc la vue est recalculé à chaque fois qu'elle est réutiliser dans une requpête, ce qui explique qu'elle ne permet pas l'optimisation.

--En revanche elle permet d'écriture une requête plus "simple".

```
--5)
CREATE MATERIALIZED VIEW vuemat_evaluation AS SELECT id_evaluation, nom, dateeval, coefficient, echelle, id_module,
AVG(note) / echelle * 20 AS moyenne_sur_20
FROM notation JOIN evaluation USING(id_evaluation)
GROUP BY id_evaluation, nom, dateeval, coefficient, echelle, id_module
LIMIT 20;
```

```
EXPLAIN ANALYSE SELECT *FROM vuemat_evaluation;
```

--Maintenant que la vue est matérialisée (donc stocké), l'utilisation de cette vue permet de gagner en performance (temps plus court).

```
--7)
SELECT * FROM vue_evaluation WHERE id_module=2;
```

```
EXPLAIN ANALYSE SELECT * FROM vue_evaluation WHERE id_module=2;
EXPLAIN ANALYSE SELECT * FROM vuemat_evaluation WHERE id_module=2; --Beaucoup plus rapide
```

```
--9)
SELECT id_module, intitule, COUNT(*) "NB. eval."
FROM evaluation Join module USING(id_module)
GROUP BY id_module, intitule;
```

```
--10)
SELECT id_module, intitule, array_agg(id_evaluation) "Tab. eval."
FROM evaluation Join module USING(id_module)
GROUP BY id_module, intitule;
```

```
--11)

SELECT id_module, intitule,
array_agg(ARRAY[id_evaluation::VARCHAR, nom]) "Tab. eval."
FROM evaluation Join module USING(id_module)
GROUP BY id_module, intitule;
```

```
--12)
SELECT id_evaluation, nom, id_module,
      ROUND(MAX(note/echelle*20)::NUMERIC,1) AS"max",
      ROUND(MIN(note/echelle*20)::NUMERIC,1) AS"min"
FROM notation JOIN evaluation USING (id_evaluation)
GROUP BY id_evaluation, nom, id_module;
```

```
--13)
WITH eval AS(
      SELECT id_evaluation, nom, id_module,
      ROUND(MAX(note/echelle*20)::NUMERIC,1) AS"max",
      ROUND(MIN(note/echelle*20)::NUMERIC,1) AS"min"
      FROM notation JOIN evaluation USING (id_evaluation)
      GROUP BY id_evaluation, nom, id_module
)
SELECT id_module, intitule,
      array_agg(ARRAY[id_evaluation::varchar, nom, max::varchar, min::varchar])
FROM module JOIN eval USING (id_module)
GROUP BY id_module, intitule;
```

```
--14)
CREATE VIEW NOTE AS
WITH eval AS(
      SELECT id_evaluation, nom, id_module,
      ROUND(MAX(note/echelle*20)::NUMERIC,1) AS"max",
      ROUND(MIN(note/echelle*20)::NUMERIC,1) AS"min"
      FROM notation JOIN evaluation USING (id_evaluation)
      GROUP BY id_evaluation, nom, id_module
)
```

```

SELECT id_module, intitule,
array_agg(ARRAY[ id_evaluation::varchar, nom, ROUND(max::numeric,1)::varchar, ROUND(min::numeric,1)::varchar])
FROM module JOIN eval USING (id_module)
GROUP BY id_module, intitule;

```

```

--16)
CREATE TYPE info_eval AS (
    id_evaluation INTEGER,
    nom VARCHAR,
    max REAL,
    min REAL
);

```

SEANCE 4 :

```

--1)
SELECT id_evaluation, nom, echelle,coefficient FROM evaluation LIMIT 3;

```

```

--2) FORMATER RÉSULTAT DE LA PREMIÈRE QUESTION EN FORMAT json
-- d'abord mettre

```

```

SELECT id_evaluation, ('{"nom":"' || evaluation.nom || '"} '::jsonb AS nom,
echelle, coefficient FROM evaluation LIMIT 3;

```

```

SELECT id_evaluation, jsonb_build_object ('nom', evaluation.nom) AS nom, echelle, coefficient FROM evaluation LIMIT 3;

```

nom	echelle
{"nom": "Mini-test 2 Relations"}	10
{"nom": "Mini-test 1 Ensembles"}	10
{"nom": "Contrôle Ensembles et relations"}	20

```

--3)
SELECT id_etudiant, id_module, note, echelle, coefficient FROM notation JOIN evaluation USING (id_evaluation) LIMIT 5;

```

```

--4)
SELECT id_etudiant, id_module, COUNT(note) AS "nb. notes" FROM evaluation JOIN notation USING (id_evaluation) GROUP BY
(id_etudiant,id_module) LIMIT 3;

```

```

--correction
SELECT id_etudiant, id_module, COUNT(*) AS "nb. notes"
FROM evaluation JOIN notation USING (id_evaluation)
GROUP BY (id_etudiant,id_module) LIMIT 3;

```

```

--5)
SELECT id_etudiant, id_module, intitule,
ROUND( (SUM(note / echelle * coefficient) /SUM(coefficient) * 20)::NUMERIC ,2) AS "moyenne"
FROM evaluation JOIN notation USING (id_evaluation)
JOIN module USING (id_module)
GROUP BY (id_etudiant,id_module, intitule);
-- ::NUMERIC car la fonction ROUND ne prend pas de float mais un numerique (plus précis qu'un float)

```

```

--6)
SELECT id_etudiant, id_module,
json_agg(jsonb_build_object ('nom', evaluation.nom)) AS tab_eval
FROM evaluation JOIN notation USING (id_evaluation)
GROUP BY (id_etudiant,id_module)
LIMIT 3;

```

```

--7)
SELECT id_etudiant, id_module, json_agg(
jsonb_build_object ('nom', evaluation.nom, 'note', notation.note, 'echelle', evaluation.echelle, 'coeff.', evaluation.coefficient)) AS
tab_eval
FROM evaluation JOIN notation USING (id_evaluation)
GROUP BY id_etudiant,id_module LIMIT 3;

```

```

--8)
SELECT id_etudiant, id_module, jsonb_build_object('Nom du module', intitule, 'Moyenne', ROUND( (SUM(note / echelle * coefficient)
/SUM(coefficient) * 20)::NUMERIC ,2),
'Evaluation', jsonb_agg(
jsonb_build_object ('nom', evaluation.nom, 'note', notation.note, 'echelle', evaluation.echelle, 'coeff.', evaluation.coefficient))) AS
tab_eval
FROM evaluation JOIN notation USING (id_evaluation) JOIN module USING (id_module)
GROUP BY id_etudiant,id_module, id_etudiant, intitule
LIMIT 3;

--9)
--Avec jsonb_insert
WITH bulletins as (SELECT id_etudiant, id_module, jsonb_build_object('Nom du module', intitule, 'Moyenne', ROUND( (SUM(note /
echelle * coefficient) /SUM(coefficient) * 20)::NUMERIC ,2),
'Evaluation', jsonb_agg(
jsonb_build_object ('nom', evaluation.nom, 'note', notation.note, 'echelle', evaluation.echelle, 'coeff.', evaluation.coefficient))) AS
tab_eval
FROM evaluation JOIN notation USING (id_evaluation) JOIN module USING (id_module)
GROUP BY id_etudiant,id_module, id_etudiant, intitule
LIMIT 1)
SELECT id_etudiant, jsonb_insert(
    jsonb_agg(tab_eval),
    '{0}',
    jsonb_build_object('Module complet', jsonb_agg(tab_eval))
) AS bulletin_complet
FROM bulletins
GROUP BY id_etudiant;

--Sans jsonb_insert
WITH bulletins_module as (SELECT id_etudiant, id_module, jsonb_build_object('Nom du module', intitule, 'Moyenne', ROUND(
(SUM(note / echelle * coefficient) /SUM(coefficient) * 20)::NUMERIC ,2),
'Evaluation', jsonb_agg(
jsonb_build_object ('nom', evaluation.nom, 'note', notation.note, 'echelle', evaluation.echelle, 'coeff.', evaluation.coefficient))) AS
tab_eval
FROM evaluation JOIN notation USING (id_evaluation) JOIN module USING (id_module)
GROUP BY id_etudiant,id_module, id_etudiant, intitule
)
SELECT id_etudiant, jsonb_build_onjeyct ('nom', personne.nom, 'prenom', personne.prenom, 'modules', jsonb_agg(module)) AS bulletin
FROM bulletin_module JOIN personne ON id_etduiant=id_personne
GROUP BY id_etudiant, personne.nom, personne.prenom;

--10)
CREATE VIEW bulletin_JSON AS WITH bulletins_module as (SELECT id_etudiant, id_module, jsonb_build_object('Nom du module',
intitule, 'Moyenne', ROUND( (SUM(note / echelle * coefficient) /SUM(coefficient) * 20)::NUMERIC ,2),
'Evaluation', jsonb_agg(
jsonb_build_object ('nom', evaluation.nom, 'note', notation.note, 'echelle', evaluation.echelle, 'coeff.', evaluation.coefficient))) AS
tab_eval
FROM evaluation JOIN notation USING (id_evaluation) JOIN module USING (id_module)
GROUP BY id_etudiant,id_module, id_etudiant, intitule
)
SELECT id_etudiant, jsonb_build_object ('nom', personne.nom, 'prenom', personne.prenom, 'modules', jsonb_agg(module)) AS bulletin
FROM bulletins_module JOIN personne ON id_etudiant=id_personne
GROUP BY id_etudiant, personne.nom, personne.prenom;

--11)
SELECT jsonb_pretty(bulletin) FROM bulletin json WHERE id_etudiant = 1 ;

```